JOBHELPER

JobHelper technical documentation

How the app is built, run and tested

For JobHelper on macOS
10 figures · 23 tables

Contents

1. Overview 4
Related documents 5
2. Know the tech stack 5
Overview 5
The language and the window 6
Claude 6
Documents and data 7
Reading the web and the mailbox 8
Privacy and the Mac 9
Quality 10
What is deliberately not used 11
Next steps 12
3. Install the app 12
Prerequisites 12
Install the packages 12
4. Run the app 13
Start the window 13
Run a search without the window 13
5. Use the command line 14
Protect and restore the data 15
6. Understand the structure of the code 16
The agents 17
What a search does with a posting 18
How a text is kept inside what the CV shows 18
What an email scan does with a message 18

How a run behaves at Claude's limit 19
The modules around the agents 19
How the assistant fills in a form 20
How the assistant decides what to do 23
How the window talks to the logic 25
7. Work with the data files 25
8. Test a change 27
Run the tests 27
What the tests may never touch 27
Test the window 27
9. Update the documentation 27
Take the pictures of the user guide again 28
Keep the numbers of figures and tables in order 28
Build the PDFs 28
Write in the house style 29
10. Troubleshoot problems 30

Applies to: the JobHelper source code, on macOS with Python 3.10 or newer.

This document is for people who work with the code: it explains how to install the app, how to run it from the command line, how the code is laid out, and how to test and document a change. To use the app, read the user guide, GUIDE.md.

1. Overview

JobHelper is a desktop app for macOS that finds job postings, scores them against a CV, writes an adapted CV and a cover letter for the good ones, and tracks applications and replies. It is written in Python. The window is built with Qt Quick (PySide6). All model calls go through the Claude Code program, so that they use the person's Claude subscription and never the paid API.

The work is done by five agents (Figure 1.1). A search runs the first four in order; an email scan runs the fifth.

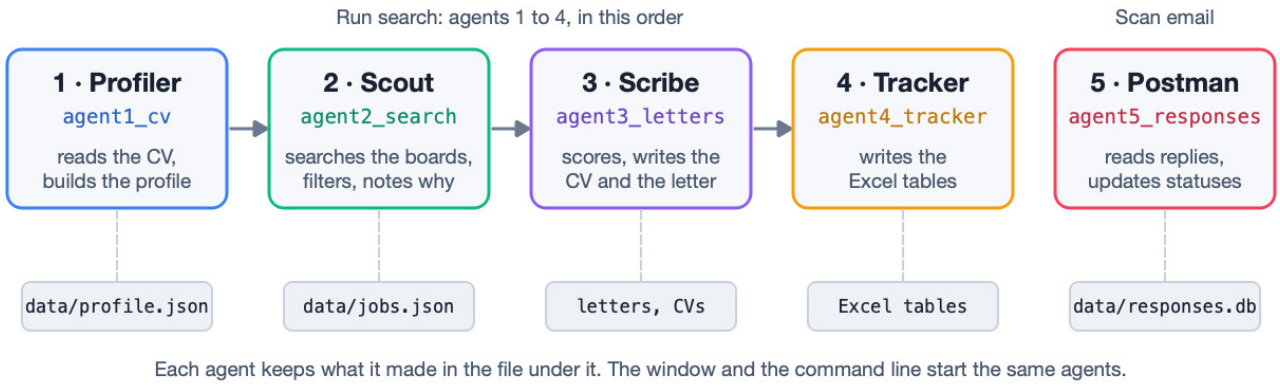


Figure 1.1 – The five agents and what each keeps

Six rules shape the whole code base, and every change must keep them (Table 1.1).

Table 1.1 – The rules every change must keep

Rule	What it means in the code
The subscription, never the paid API	llm.ask refuses to run while ANTHROPIC_API_KEY is set. No code path calls the API directly
The mailbox is read-only	Folders are opened read-only and messages are fetched with PEEK. Nothing sends, deletes, moves or marks mail
Nothing is claimed that the CV does not show	Every written text is checked in plain code against the CV after the model wrote it
The person's own letters are never changed	Only letters the app wrote may be replaced or moved
Nothing is deleted	Data is backed up before a bulk change; documents are moved to an encrypted backup, old copies to the Trash
No personal data in the code	Names and facts of a real person never appear in code, tests or documents

Related documents

Table 1.2 lists the other documents of the project.

Table 1.2 – The documents of the project

Document	Use it for
GUIDE.md	Using the app, for someone who never opens a terminal
COMMANDS.md	The full reference of every command and option
HANDBOOK.md	The rules the app is built by, decisions already made, what is left to do
CHANGELOG.md	What changed, day by day. The app shows it under **Help**

2. Know the tech stack

Overview

JobHelper is built from a small number of well-known parts, each chosen for a reason that follows from what the app must do: run on one person's Mac, keep their

data private, and use their Claude subscription instead of a paid API. This chapter names each part and says why it was chosen, so that a change does not replace one without knowing what it was there for.

The language and the window

Table 2.1 lists what the app itself is written in.

Table 2.1 – The language and the window

Technology	Used for	Why it was chosen
Python 3.10 or newer	All the logic	The libraries for Word files, Excel files, mail and web pages are mature in Python, and the official Claude library is a Python library. One language for everything keeps the project small enough for one person
Qt Quick (QML) with PySide6	The window	It draws a native window on the graphics card, so the robots and the lists animate smoothly, and it needs no web browser inside the app. An earlier window built from a web page was removed: two front ends had to be kept the same, and the web one was slower and looked foreign on macOS
QML for layout, Python for logic	The split between ui_qt/ and jobhelper/	A QML file only shows things and calls the bridge; it never reads a file. The logic can therefore be tested without a window, and the window can be tested without real data
Threads with Qt signals	Searches, scans and rewrites	Long work must not freeze the window. A worker thread does it and answers through a signal, which Qt delivers on the window's own thread

Claude

Table 2.2 lists how the app reaches the model.

Table 2.2 – How the app reaches Claude

Technology	Used for	Why it was chosen
claude-agent-sdk driving the Claude Code program	Every question to the model	It uses the person's Claude subscription. The paid API bills per use and would make every search cost money; that is why the app refuses to run while ANTHROPIC_API_KEY is set
One module, llm, for all calls	Asking, counting usage, noticing the limit	With a single door, the rules hold everywhere: no call while the limit is used up, every call counted, no key in an error message
Plain code before and after the model	Filters before scoring; checks after writing	A check in code is free, instant and always gives the same answer. The model is asked only what code cannot decide, and what it wrote is checked against the CV in code

Documents and data

Table 2.3 lists what reads and writes the person's files.

Table 2.3 – Documents and data

Technology	Used for	Why it was chosen
python-docx	Reading a CV; writing the adapted CV and the letter	Employers expect Word files. Editing the person's own .docx keeps its fonts and layout, where generating a new file would lose them
pypdf	Reading a CV that is a PDF; joining the letter and the CV into one PDF	It is pure Python, so nothing else has to be installed
LibreOffice (optional)	PDF copies of Word files	It converts a Word file faithfully and is free. It is optional: without it the app still works and offers Word files only
openpyxl	The Excel tables	People already track applications in Excel; the tables can be opened, sorted and shared without the app
JSON files	The job list, the profile, the Options	The data of one person is small. A file can be read whole, backed up by copying, and repaired by hand if it must be
SQLite	The replies read from the mailbox	Replies are many, are looked up by message, and must never be read twice. A small database does that; it is part of Python and needs no server

Reading the web and the mailbox

Table 2.4 lists what reaches outside the Mac.

Table 2.4 – Reading the web and the mailbox

Technology	Used for	Why it was chosen
requests with beautifulsoup4 and lxml	Reading the job boards and company career sites	The boards have no common interface, so their public pages are read as a browser would read them, at a modest pace. No search needs a key or an account
imaplib, from Python itself	Reading replies	IMAP works with almost every mail provider and can open a mailbox read-only, which is the promise the app makes: it never sends, deletes or marks mail
An app password instead of a sign-in window	Access to the mailbox	It can be cancelled at the provider at any time without changing the real password, and it needs no registration of the app with each provider

Privacy and the Mac

Table 2.5 lists what protects the data and ties the app to macOS.

Table 2.5 – Privacy and the Mac

Technology	Used for	Why it was chosen
AES-256-GCM, from cryptography	Encrypting the data files and the backups	Applications and replies are private. This method also notices when a file was changed, so a damaged or tampered file is refused instead of read
The macOS Keychain, through the security program	Keeping the data key and the mailbox passwords	Secrets are then protected by the Mac's login and are never written to a file of the project
Signed rule files	The rules for CVs and letters, the letter template, the master CV	These files decide what is written in the person's name. A change made outside the app must be approved before the next run
A LaunchAgent	The weekly run	It is how macOS starts a task on a schedule, also when the window is closed
Apple events, through the osascript program	The assistant reads and fills in the form that is open in the person's browser (Chrome, Brave, Edge or Safari)	The form is filled in where the person is already signed in, with nothing to install. Chrome and macOS each ask for consent first, and the app can only do what a script in the page can do: it cannot attach files or read saved passwords
Hash-pinned packages in requirements.lock	Installing	Each package is checked against a fingerprint, so a changed package cannot be installed unnoticed

Quality

Table 2.6 lists what keeps the code correct.

Table 2.6 – Tools for quality

Technology	Used for	Why it was chosen
pytest	More than 1,300 tests	Every change comes with tests. The suite needs no network and never touches real data, so it can be run at any time
The window loaded without a screen	Tests of the window	The real QML is loaded and driven, so a broken binding or a missing name fails a test instead of reaching the person
pyflakes	Unused and undefined names	It finds a whole kind of mistake in a second, before a test has to
Git, kept on the Mac only	The history of changes	Every finished step is a commit that can be gone back to. There is no remote copy, because the project holds a real CV

What is deliberately not used

Table 2.7 lists parts that were considered and left out.

Table 2.7 – What is deliberately not used

Not used	Why not
The paid Claude API	It bills per use. The subscription the person already has does the same work
A web browser inside the app	It was tried and removed: slower, larger, and a second front end to keep the same
A browser extension for filling in forms	It would have to be installed by hand and kept in step with the app. Apple events reach the same page without one
A database server	One person's data fits in files; a server would be one more thing to install and to keep running
A cloud service for the data	Applications, replies and the CV stay on the person's Mac
Sending mail	The app reads replies and drafts texts. The person sends them, so nothing leaves in their name without them

Next steps

- Install the app

3. Install the app

Prerequisites

Before you install, make sure you have:

- macOS.
- Python 3.10 or newer.
- The Claude Code program, signed in with a Claude subscription: run claude auth login once.
- Optional: LibreOffice, for PDF copies of the documents.

> **Important**
> Do not set ANTHROPIC_API_KEY. Leaving it unset is what makes the app use the subscription, and the app refuses to run a search while it is set.

Install the packages

1. Open a terminal in the project folder.
2. Create a virtual environment and turn it on:

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

3. Install the exact, hash-checked packages:

```
pip install --require-hashes -r requirements.lock
```

4. Optional: copy the example settings file. The mailbox password never goes into it.

```
cp .env.example .env
```

Table 3.1 lists the main packages and what each is for.

Table 3.1 – The main packages

Package	Used for
claude-agent-sdk	Driving the Claude Code program, which uses the subscription
PySide6	The window (Qt Quick and QML) and the PDF of the documentation
python-docx, pypdf	Reading a CV; writing CVs and letters as Word files
openpyxl	The Excel tables
requests, beautifulsoup4, lxml	Reading the job boards
cryptography	AES-256-GCM encryption of the data files and the backups
pytest	The tests. pyflakes, installed separately with pip install pyflakes, checks for unused or undefined names

4. Run the app

Start the window

1. Turn the virtual environment on.
2. Start the app:

```
python app_qt.py
```

3. Follow the welcome card: sign in to Claude, choose a CV, and connect a mailbox if you want replies to be read. The user guide describes each step.

Run a search without the window

A search can also be run from the command line. It does the same work as **Run search** and writes to the same files.

```
python run.py
```

Table 4.1 lists the options used most often.

Table 4.1 – The most used options of run.py

Option	What it does
--where CITY	Searches in another city than the one saved in Options
--source NAME	Chooses the job boards: czech, linkedin, remote, adzuna, jooble or all
--include-remote	Adds the remote boards
--threshold N	Writes documents only from this score
--per-query N	Asks each board for this many postings per role
--all-languages	Keeps postings written in other languages than English
--max-age-days N	Drops postings older than this
--reuse-profile	Does not read the CV again
--skip-search	Scores the postings already in the list, without searching

> **Note**
> Do not run a search from the command line while the window is running one. Both write the same job list.

5. Use the command line

Everything the window does with applications and replies can also be done with status.py. Table 5.1 lists its commands.

Table 5.1 – The commands of status.py

Command	What it does
python status.py list	Lists every tracked job with its id and status
python status.py apply ID	Marks a new job as applied
python status.py update ID STATUS	Sets the status of a job
python status.py sync	Reads replies in the mailbox and updates statuses. --days N chooses how far back
python status.py responses	Shows the numbers of replies and rejection reasons
python status.py followups	Lists applications with no reply for a number of days
python status.py rebuild	Writes the Excel tables again
python status.py retailor ID	Writes a letter and an adapted CV for one job
python status.py pdf	Makes PDF copies of the documents (needs LibreOffice)
python status.py letters	Takes in letters the person wrote themselves
python status.py prune	Removes new jobs you have already applied to
python status.py repair-emails	Reads email text again, without Claude, and fixes replies that were misread because the email was HTML only
python status.py repair-statuses	Fixes jobs whose status the app does not know
python status.py usage	Shows the Claude usage of the app, in total and per kind of work
python status.py security ACTION	Encryption, Keychain and backup tools (Table 5.2)

Protect and restore the data

The data files are encrypted with a key kept in the macOS Keychain. Table 5.2 lists the actions of python status.py security.

Table 5.2 – The actions of status.py security

Action	What it does
status	Says whether encryption is on and whether a watched file changed
recovery-key	Prints the recovery key. The window shows the same key in the **Account** card
restore-key	Puts a saved recovery key back into the Keychain
approve	Accepts a change to a rule file, the letter template or the master CV as trusted
backup, backup-all	Makes an encrypted backup of the data, or of everything
restore, test-restore	Restores a backup, or proves that a backup can be restored

> **Warning**
> Anyone who has the recovery key can read the data. Never print it into a log, a chat or a file that is kept.

6. Understand the structure of the code

The code is in three places (Table 6.1).

Table 6.1 – The folders of the project

Folder	What is in it
jobhelper/	All the logic: the agents, the rules, reading and writing files
ui_qt/	The window: one QML file per window or part of the main screen
tests/	The tests, and the script that takes the pictures of the user guide
scripts/	Tools for the documentation: the PDF builder, the numbering check, the diagram
docs/	The user guide, this document, their PDFs and the pictures
data/	The person's data. Never part of the code; see section 7

The agents

Table 6.2 names the module of each agent and what it may ask Claude.

Table 6.2 – The five agents

Agent	Module	What it does	Asks Claude
Profiler	agent1_cv	Reads the CV and builds the profile and the search roles	Once per CV
Scout	agent2_search	Searches the boards, filters, keeps every dropped posting with its reason	Never
Scribe	agent3_letters	Scores postings five per call; writes the adapted CV and the letter	Yes
Tracker	agent4_tracker	Writes the Excel tables	Never
Postman	agent5_responses	Reads replies and files each to its application	For new replies only

What a search does with a posting

Figure 6.1 follows one posting through a search. Everything before the scoring is plain code: a posting that does not fit the rules, or is plainly another profession, never costs a call to Claude, and it is kept with the reason so that the person can bring it back.

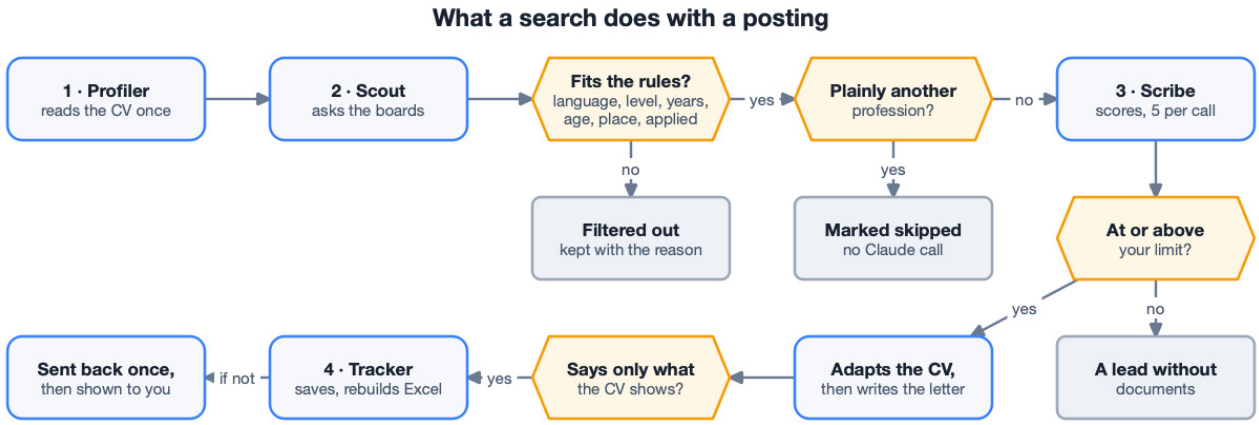


Figure 6.1 – What a search does with a posting

How a text is kept inside what the CV shows

An adapted CV, a letter, a form answer and an interview answer are all written by Claude and then checked in code (Figure 6.2). A prompt that says "do not invent" is not a guarantee, so the draft is read for technologies the CV does not show and for terms of the posting stated as experience. What is found is sent back once; what is still there is kept and shown to the person, never cut out silently.

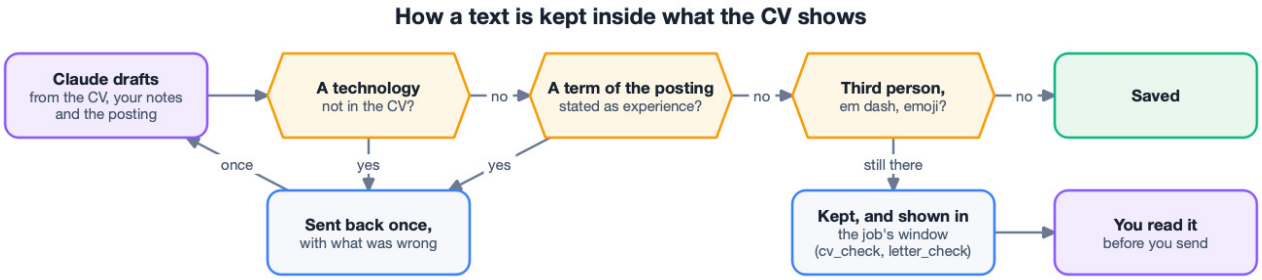


Figure 6.2 – How a text is kept inside what the CV shows

What an email scan does with a message

Figure 6.3 shows the way of one message. The mailbox is opened read-only, and rules decide without a call which messages are worth a question to Claude.

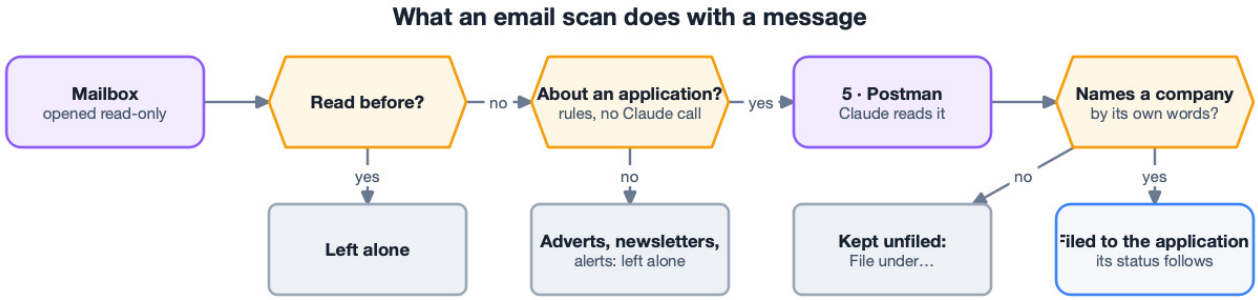


Figure 6.3 – What an email scan does with a message

How a run behaves at Claude's limit

The limit of the subscription is a state of a run, not an error (Figure 6.4). Every call goes through llm.ask, which refuses to call when a limit is used up; the run saves its work, shows the reset time, and goes on from the same job when the person presses **Resume**.

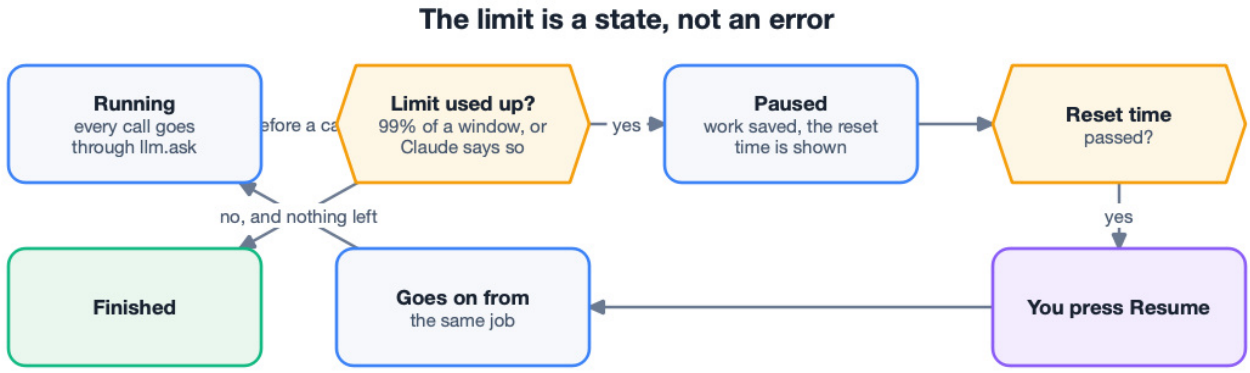


Figure 6.4 – The limit is a state, not an error

The modules around the agents

Table 6.3 groups the other modules by what they are for.

Table 6.3 – The modules around the agents

Group	Modules	What they do
Reading postings	scrapers_cz, scrapers_intl, scrapers_remote, scrapers_ats, scrapers_page	One module per kind of source. A site that turns programs away is left alone
Rules without a model call	filters, prefilter, language, places, keywords, salary, merge, followup, planner	Level, years, language, location, profession, pay, duplicates, reminders

Continuation of Table 6.3

Group	Modules	What they do
Writing	cv_tailor, cover_letter, letters, form_answers, interview_prep, prep_doc, pdf	The adapted CV, the letter, form answers, the interview sheet
Mail	email_client, mailhosts	Reading mailboxes read-only; which server a mailbox is read from
The assistant	assistant, assistant_brain, assistant_notes, details, chrome, form_fill, form_choices, answer_memory	The fixed requests and the facts about the app; what Claude makes of a free request, checked before anything happens; the person's own details; the link to Google Chrome; reading a form, deciding what each field is, and filling it in; choosing from lists and menus; the person's own wording, remembered
Claude	llm, usage	The one place that asks Claude; counting usage and knowing the limits
Storage and safety	store, db, vault, integrity, housekeeping, health, trouble	The job list, the reply database, encryption, signed rule files, backups, the start-up check, the log of errors
The window's back end	ui_api, api_documents, api_interview, api_letters, api_assistant, api_talk, qt_bridge, bridge_runs, bridge_jobs	What the window calls. Api holds the logic; Bridge exposes it to QML and runs long work in threads

How the assistant fills in a form

The assistant works on the tab that is in front in the person's browser (Figure 6.5), in steps that are each small enough to check, all in form_fill:

1. A script lists the fields of the page: what each is called, its kind, and whether it already holds something. It numbers them with an attribute and sends back no value the person typed.

2. plan() decides in plain code what each field is. The person's details come from details; open questions go to form_answers in one call; everything else is left, with the reason.
3. A second script writes the values into fields that are still empty and outlines them.
4. Lists, radio buttons and menus the page draws itself are choices: form_choices decides them, and a third script opens a menu, reads its entries, and presses the chosen one.

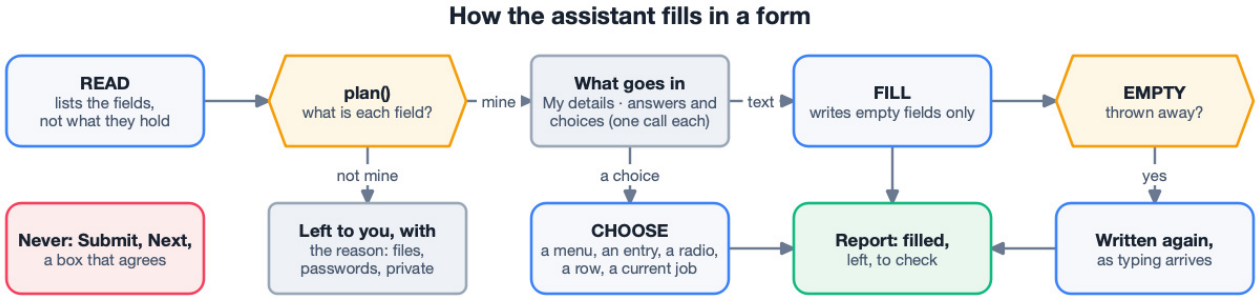


Figure 6.5 – How the assistant fills in a form

Table 6.4 gives the rules these steps keep. tests/test_assistant.py checks each of them.

The whole request is also tried in a test area: tests/fixtures/fill_in_browser.py serves the forms in tests/fixtures/forms/ from this machine, opens one in a browser engine, and runs the same code the window runs, with Claude replaced by prepared answers and the data in a temporary folder. There is a plain form, one built the way component libraries build them (each box keeps its own value and undoes what did not arrive like typing; its lists are menus that hold nothing until they are opened), a long one with lists, radio buttons and a number, and one that sits in a frame from another address. To check a kind of form the assistant gets wrong, add a page there and a test that names what must be filled and what must be left.

Table 6.4 – The rules of filling in a form

Rule	How it is kept
The form is never sent	No script presses a button that sends. The one script that presses anything (CHOOSE) accepts five kinds of element: what opens a menu, an entry of a menu, a radio button, a button whose words say that it adds a row for a job or a degree, and a tick box whose words say that a job goes on. It reads those words again before it acts, and refuses anything that sends, continues, uploads or agrees. A test reads the scripts for those calls, and the test area checks on every form that nothing was sent
What a page throws away is noticed	After writing, EMPTY says which fields hold nothing; those are written again the way typing arrives; what is still empty is reported to the person, not counted as filled
A choice is made only when something settles it	form_choices.settle uses the person's details and the job; one call to Claude covers the rest, and an answer that is not one of the field's own options is dropped. Every choice is marked for checking
Only the assistant's own answers are read back	To learn the person's corrections, VALUES is asked for the numbers of the fields the assistant filled as answers, and for nothing else
Only empty fields are written	The filling script tests each field again just before it writes
Values are data, never code	They travel as JSON inside the script, and the script travels as an argument of osascript
What the page answers is not trusted	read_answer and fill_answer accept only the expected shape and cut every text to a fixed length
Private fields are left alone	Passwords, files, payment and personal-status fields are recognised by their kind or their label and never planned

Continuation of Table 6.4

Rule	How it is kept
The right page gets the values	The filling script compares the page's address with the one that was read, and writes nothing when they differ
Chrome is never started	The request first asks whether Chrome is running and stops there when it is not

How the assistant decides what to do

Figure 6.6 shows what happens to a request.

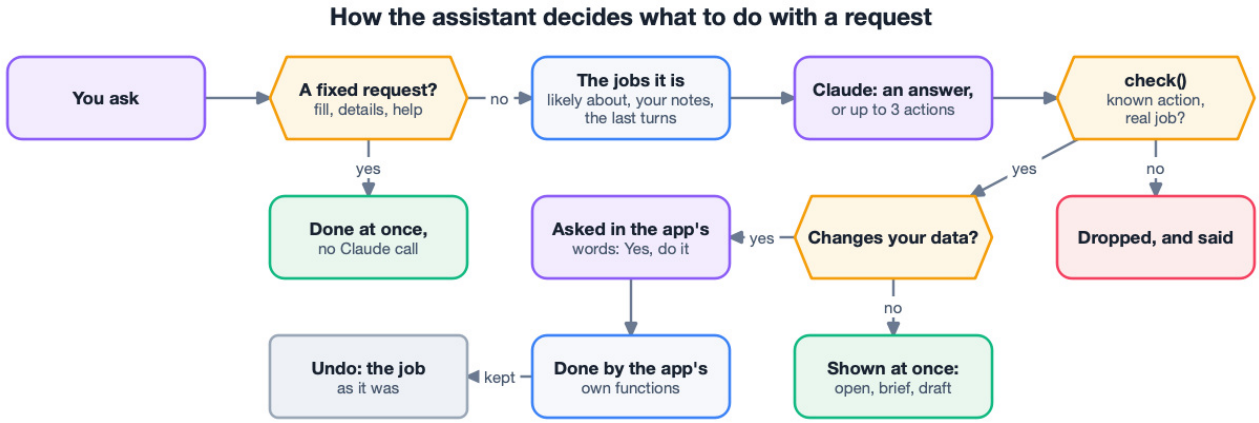


Figure 6.6 – How the assistant decides what to do with a request

A request that is not one of the fixed ones ("fill in the form", "my details", "help") goes to assistant_brain.decide: one call to Claude, with the jobs as lines, the numbers, the last turns of the talk and the list of actions. Claude answers in words or names one action. Table 6.5 gives the rules that keep this safe; api_talk carries them out.

Table 6.5 – The rules of acting on a request

Rule	How it is kept
Claude does nothing itself	It returns JSON. check keeps only words, up to three actions from a fixed list, and arguments of the right kind; an unknown action or a job that is not in the list is dropped
Only the jobs a request is likely about are sent	relevant picks the job in front, the companies named in the talk, what is open and the best leads, up to 70; closed applications come only when the request is about them. Sending every job with every request cost three times as much
What was agreed to can be taken back	assistant_confirm keeps the job as it was, and any lead that marking it applied folded away; assistant_undo puts them back. Not offered while a search is running
The prompts were tried against the real Claude	A trial on invented data showed a reply's date being read as an interview's date, and a guessed reason for a score. The job lines were reworded ("an invitation arrived", "no date set") and the rules tightened; run such a trial again after changing a prompt
Nothing of the person's is changed without their yes	An action that changes data comes back to the window as a question, worded by the app from the checked arguments, not by Claude. assistant_confirm carries it out and checks it once more
A rewrite is shown before it is saved	The new paragraph or answer is made first and travels with the question; saving is the confirmed action
An answer comes from what the app holds	The prompt carries the jobs, their state and the facts about the app; a brief, advice, a review and practice feedback each have their own prompt with only their own facts
The fixed requests cost nothing	They are recognised by their words before any call is made

Continuation of Table 6.5

Rule	How it is kept
Initiative uses no model	The briefing and the reminders are computed from the data; noticing a form is off until it is turned on, and looks only at the address and title of the tab in front and the number of its empty fields

How the window talks to the logic

Figure 6.7 shows the way of one press of a button through the layers and back.

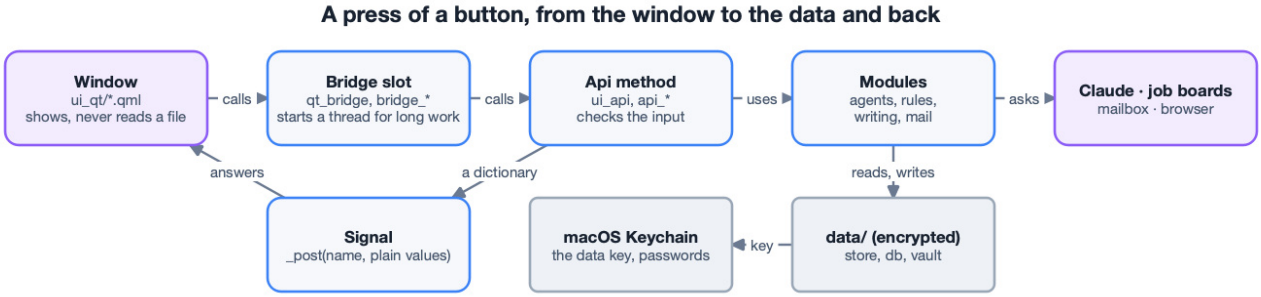


Figure 6.7 – A press of a button, from the window to the data and back

The window never touches a file. Every action goes through three layers:

1. A QML file in ui_qt/ calls a slot of bridge, for example bridge.saveOptions(...).
2. The slot, in qt_bridge.py, bridge_runs.py or bridge_jobs.py, calls a method of Api.
3. The method, in ui_api.py or one of the api_*.py files, checks the input, does the work and returns a plain dictionary: {"ok": True, ...} or {"ok": False, "msg": "why"}.

Long work (a search, a scan, a rewrite) runs in a thread and answers through a signal.

> **Important**
> Every input is checked twice: in the window, where the person sees a red message next to the field, and again in the Api method, so that nothing unusable is saved whatever sent it.

7. Work with the data files

Figure 7.1 shows how the data is kept safe: every read and write goes through vault, the key lives in the Keychain, a backup is made before a bulk change, and a rule file changed outside the app must be approved.

Everything a person makes with the app is in data/ (Table 7.1). The folder is not part of the code and is never committed.

Follow these rules when code changes data:

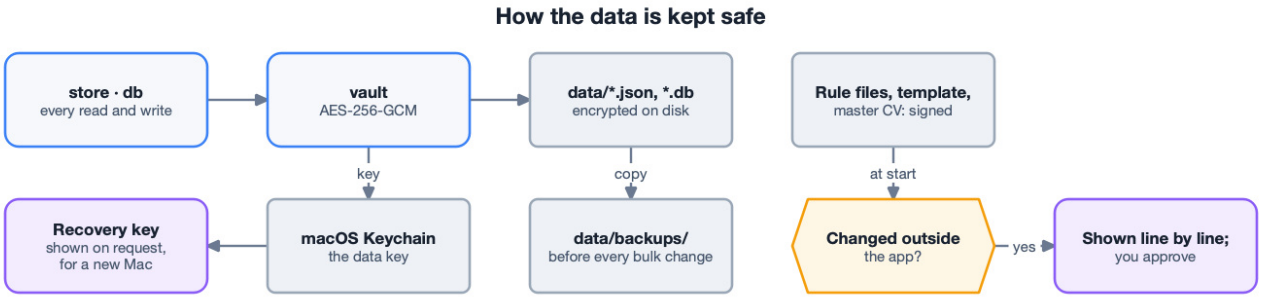


Figure 7.1 – How the data is kept safe

Table 7.1 – The files and folders in data

Path	What it holds	Encrypted
data/jobs.json	Every posting and application	Yes
data/profile.json	The profile read from the CV	Yes
data/ui_settings.json	The Options	Yes
data/responses.db	Every reply that was read	Yes
data/usage.json	The Claude usage and the last known limits	No
data/backups/	Backups, made before any bulk change	Yes
data/cv/, data/my_letters/	The master CV and the person's own letters	No, so that Word can open them
data/tailored_cvs/, data/cover_letters/	The documents the app wrote	No, so that Word can open them
data/app_trouble.log	One line for each error the app carried on after	No

1. Read and write the job list only through store, the replies only through db, and any encrypted file only through vault.
2. Call vault.make_backup(label) before a change that touches many jobs.
3. Move a document to the encrypted backup instead of deleting it; put old copies in the Trash.
4. Do not change the data while the window is running.

8. Test a change

Run the tests

1. Run the whole suite. It takes about two minutes and needs no network:

   ```
   python -m pytest -q
   ```

2. Check for unused and undefined names:

   ```
   python -m pyflakes jobhelper tests scripts
   ```

Both must pass before a change is committed.

What the tests may never touch

The tests work in temporary folders with stand-ins. Table 8.1 lists what they never reach and what stands in for it.

Table 8.1 – What the tests never touch

Never touched	What stands in
The real data/ folder and .env	Temporary folders, set by the isolate_data fixture
The macOS Keychain	An in-memory keychain
Claude	A function that returns a prepared answer
Job boards and mail servers	Functions that return prepared postings and messages
Notifications, Finder, the Trash, the browser	Functions that only record that they were called
The person's Google Chrome	A guard that fails the test. The scripts that read and fill in a form run in a browser engine on a local page, tests/fixtures/forms/

Test the window

The window is loaded without a screen and driven by the tests in tests/test_ui_qml.py. Use the helpers of its ui fixture: ui.js(code) runs JavaScript in the window, ui.find(name) finds an item by its objectName, and ui.wait(ms) lets the window run.

> **Tip**
> After a change that can be seen, take a picture of the window and look at it. A test that reads a text does not notice that the text is not drawn.

9. Update the documentation

Each change that a person can see needs three things: a line in CHANGELOG.md, the

user guide brought up to date, and its pictures taken again if a pictured window changed.

Take the pictures of the user guide again

The pictures show the real window filled with invented people and companies. A script takes them:

```
python -m pytest tests/guide_shots.py -q -o addopts=""
```

Keep the numbers of figures and tables in order

Figures and tables are numbered inside their chapter, as in a thesis: the second figure of chapter 4 carries the number 4.2. Every picture has a caption under it and every table a caption above it: a line in italics made of the word Figure or Table, the number, an en dash and the title. The text refers to each by its number.

1. Write the caption of a new figure or table with any number that is not used yet, for example 900.
2. Renumber the captions and every reference in one go:

   ```
   python scripts/guide_number.py
   ```

Build the PDFs

Figure 9.1 shows how a PDF is made from its text.

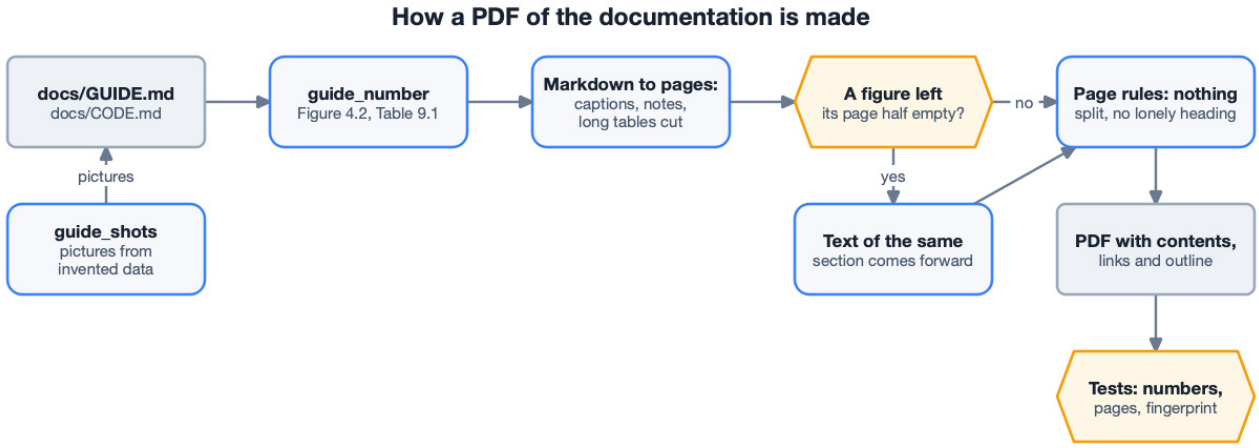


Figure 9.1 – How a PDF of the documentation is made

```
python scripts/guide_pdf.py
```

The script makes docs/GUIDE.pdf and docs/CODE.pdf. It lays each one out like a thesis: a title page, the contents with dotted lines that lead to the page numbers, then the text with its page number at the top right. The entries of the contents, references to figures and tables, and links to another chapter can be selected and lead to their place. A caption stands centred under its picture and above its table. There are no lists of figures and tables. The script keeps the page rules in Table 9.1. A test fails while a PDF is older than its text.

Table 9.1 – The page rules of the PDFs

Rule	How the script keeps it
A figure is never split across two pages	A picture and its caption are one block; a block that would cross a page starts on the next one
A table stays on one page	The same, with the caption above it
No page is left half empty because a figure or a table had to start on the next one	The text that follows it in the same section is brought forward to fill the space; a figure, a table or a note comes forward only if all of it fits. Text never crosses a heading. A table longer than a page begins where its page has room, and a picture that almost fits is shown a little smaller
What names a place leads to it	A PDF library adds a link for every entry of the contents, every mention of a figure or a table, and every link to another chapter, and writes the chapters into the PDF's outline
The points of a list stand out	A heavy square before each point, a bold number before each step
A table longer than a page continues between two rows	It is cut into parts that fit, each under the line "Continuation of Table" with its number, and with the heading row repeated
A heading is never left alone at the bottom of a page	A heading whose text would start on the next page moves there too

Write in the house style

Table 9.2 gives the rules for the text of both documents.

Table 9.2 – The rules for writing the documentation

Element	Rule
Headings	Start with a verb and name the task: "Connect your mailbox"
Steps	Numbered, one action each, in the order they are done
Reference material	In a table, with a caption and a reference from the text
Names of buttons and labels	Bold, spelled exactly as in the app
Commands and file names	Monospace. The user guide contains no commands
Links	The link text says where it leads, never just "here"
Notes	Four kinds: Note, Tip, Important, Warning, as a quoted block that starts with the bold word

10. Troubleshoot problems

Table 10.1 lists the problems met most often while working on the code.

Table 10.1 – Problems while working on the code

What you see	Cause	What to do
"Unset ANTHROPIC_API_KEY to use your subscription"	The paid key is set in the shell	Run unset ANTHROPIC_API_KEY, then start again
The window says Claude is not signed in	The Claude Code program has no session	Run claude auth login
A test of a PDF fails after a change to a document	The PDF is older than its text	Run python scripts/ guide_pdf.py
A test of the numbering fails	A figure or table was added or moved	Run python scripts/ guide_number.py
A QML property named onSomething does nothing	QML reads such a name as a signal handler	Give the property another name
An item made by a Repeater is not found in a test	It is not a child in the object tree	Use ui.seen(part) or ui.drawn(name)
The window stutters while a thread works	Python runs one thread at a time	Give the thread short rests, as keywords.read_ahead does
Data looks wrong after an experiment	A change was made to the real data	Restore the newest backup: python status.py security restore